DEPI · FULL-STACK .NET WEB DEVELOPMENT · ROUND 5

Lumora Learning Platform

Requirements Gathering & Analysis

TRACK	Full-Stack .NET Web Development – Round 5 (DEPI)	PARTNER	AST – CAI5_SWD5_S6
TEAM	Code Raiders	DATE	October 16, 2026

1. Stakeholder Analysis

This section defines the primary individuals, groups, and external systems that interact with or are affected by the Lumora Learning Platform.

1.1 Internal Stakeholders

Stakeholder	Responsibilities & Goals
Student (Primary consumers)	Create accounts, search/filter courses, enroll (free/paid), manage cart, apply coupons, watch videos, solve quizzes (with explanations), track progress, take notes, view purchase history, download/verify certificates, share on LinkedIn, and rate/review courses.
Instructor (Content creators)	Perform CRUD on courses/sections/lessons, upload videos, set prices/discounts/difficulty, view enrollment/revenue stats, create varied quizzes, answer Q&A, manage announcements, and preview/draft courses.
Admin (Platform Owner)	Manage all users, review and approve courses for publishing, manage categories/tags, configure platform settings, monitor sales/revenue, and track analytics (best-selling, completion rates, engagement).

1.2 External Stakeholders (Systems & Services)

System / Service	Role in the Platform
Payment Gateway (Stripe)	Processes secure payments for paid courses, handles confirmation, and returns real-time payment status to the platform.
Cloud Hosting & Storage (Microsoft Azure)	Hosts the application (Azure App Service), stores media/videos (Azure Blob Storage), delivers content efficiently (Azure CDN), and ensures secure streaming via signed URLs.
Notification Service (Optional)	Handles transactional emails including registration confirmation, payment receipts, course announcements, and certificate issuance alerts.

2. User Stories & Use Cases

This section details the system's use cases mapped to each actor, including dependencies such as <<include>> and <<extend>> relationships based on the finalized Use Case Diagram (see Section 2.5).

2.1 Common & Guest Use Cases

- **Actors:** Guest, Registered User (Abstract class inherited by Student, Instructor, and Admin).
- **Shared Operations:** Register, Login, Logout, Manage Profile, Browse & Search Courses, View Course Details, Verify Certificate (Public feature).

2.2 Student Use Cases

Category	Use Cases & Relationships
Enroll & Purchase	Enroll in Free Course, Purchase Course (Cart & Checkout). Note: Purchase Course <<includes>> Process Payment (via Stripe) and <<extends>> Apply Coupon.
Learning & Progress	Watch Course Lessons (<<includes>> Stream Video via Microsoft Azure). Track Learning Progress (<<includes>> Mark Lesson Complete, which <<extends>> Generate Certificate).
Interaction & Assessment	Take Notes / Bookmark Lessons, Take Quizzes, Ask Question (Q&A).
Feedback & Certification	Rate & Review Courses (<<extends>> Vote Review as Helpful). View My Certificates (<<extends>> Share on LinkedIn).

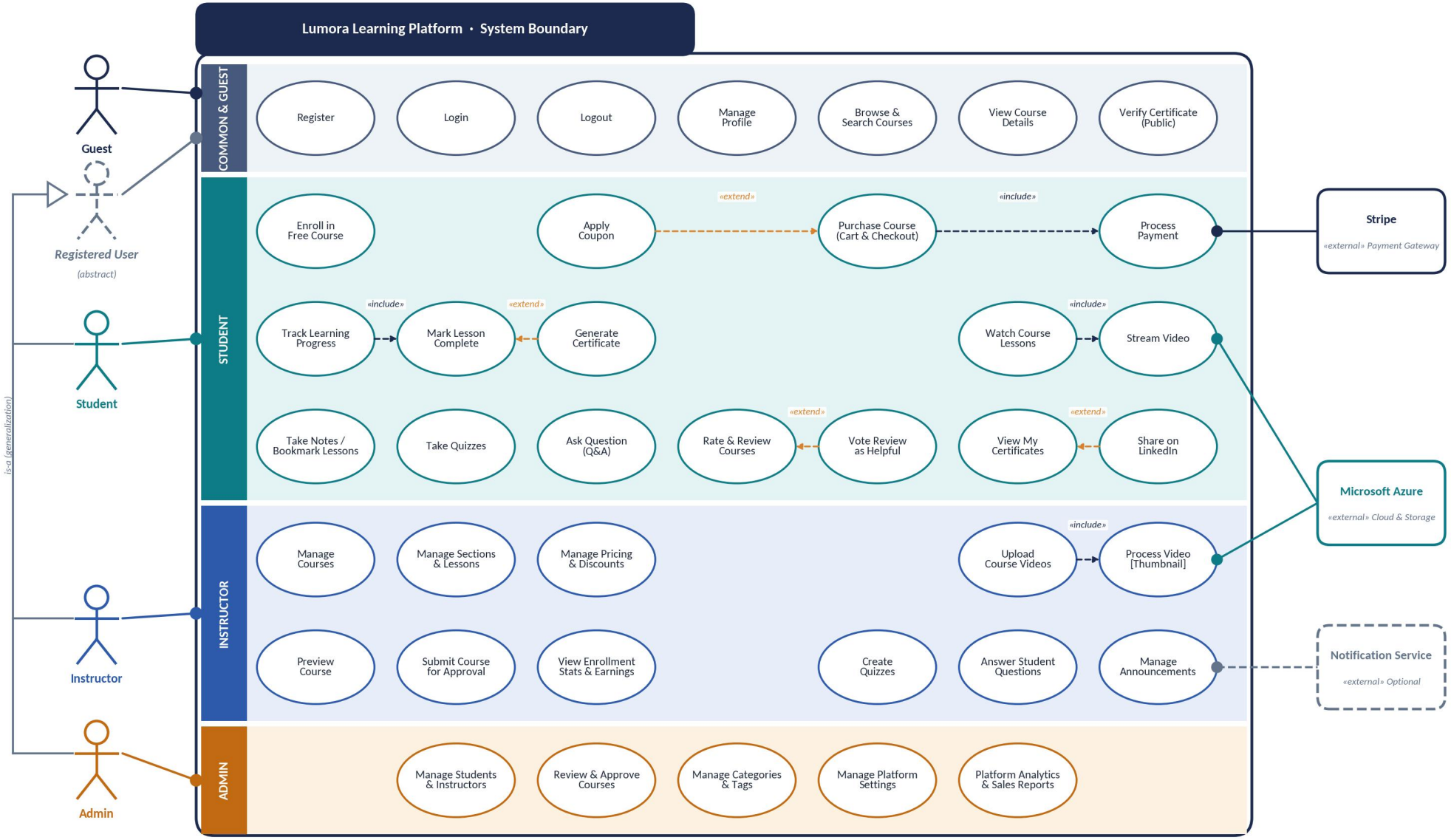
2.3 Instructor Use Cases

Category	Use Cases & Relationships
Course Authoring	Manage Courses, Manage Sections & Lessons, Upload Course Videos (<<includes>> Process Video [Thumbnail] via Azure). Manage Pricing & Discounts.
Publishing & Monitoring	Preview Course, Submit Course for Approval, View Enrollment Stats & Earnings.
Student Interaction	Create Quizzes, Answer Student Questions, Manage Announcements.

2.4 Admin Use Cases

Category	Use Cases & Relationships
Platform Management	Manage Students & Instructors, Review & Approve Courses, Manage Categories & Tags, Manage Platform Settings, view Platform Analytics & Sales Reports.

2.5 Use Case Diagram



Legend: - - - → **«include»** (mandatory sub-behavior) - - - → **«extend»** (optional extension) **Hollow arrow** (actor generalization) **Dashed box** (optional external system)

3. Functional Requirements

Detailed functional specifications based on the core interactions of the three primary roles within the system. Constraints and edge cases are integrated into the feature definitions.

3.1 Student Module

- **Authentication:** Create an account with email verification and secure login.
- **Course Discovery:** Browse, search using filters, and categorize available courses.
- **Enrollment & Finance:** Shopping cart functionality, secure Stripe checkout, apply promo codes/discounts. Support for both free and paid courses. **Constraint:** A student cannot buy the same course twice; access is strictly denied if payment verification fails.
- **Learning Experience:** Custom video player with playback controls. Ability to take notes and bookmark specific lessons.
- **Progress Tracking:** Auto-finish marker for completed lessons and dynamic progress bar for overall course completion.
- **Assessments:** Attempt end-of-section/course quizzes with varied question types, display immediate results with explanations, enforcing minimum passing score requirements.
- **Certification:** Generate unique PDF certificates (via iTextSharp) displaying student name, course title, completion date, and a unique verification ID. Support direct sharing to LinkedIn.
- **Feedback System:** 5-star rating system, written reviews, display average ratings, and helpfulness voting for reviews.

3.2 Instructor Module

- **Authentication:** Instructor account creation with email verification.
- **Course Creation:** Build courses via a drag-and-drop interface. Divide content into sections and lessons. Save as draft and support bulk file uploads.
- **Content Management:** Upload videos to Azure with automatic processing, compression, and thumbnail generation. Secure streaming via Signed URLs.
- **Course Metadata:** Add descriptions, set pricing, apply discounts, and assign difficulty levels. Set up payout methods.
- **Course Maintenance:** Perform CRUD operations on courses and content. Respond to students via the Q&A section and broadcast course announcements.
- **Assessments:** Create comprehensive quizzes supporting multiple-choice, true/false, and fill-in-the-blank questions.
- **Dashboard:** View detailed student enrollment statistics and revenue tracking.

3.3 Admin Module

- **User Management:** Full access to manage, suspend, or block users (Students & Instructors).
- **Quality Assurance:** Mandatory review and approval workflow for all newly uploaded courses prior to publishing.
- **Taxonomy:** Categorize and tag courses for platform organization.
- **Platform Configuration:** Update platform settings, names, icons, and review financial transactions.
- **Advanced Analytics:** Visualize key metrics including best-selling courses, course rating averages, instructor earnings, student engagement metrics, and monthly revenue tracking.

4. Non-Functional Requirements & Technology Stack

Category	Technologies & Specifications
Backend Framework	ASP.NET Core MVC, Entity Framework Core (ORM).
Security & Identity	ASP.NET Identity for authentication and Role-Based Access Control (RBAC).
Cloud Infrastructure	Microsoft Azure App Services (Hosting).
Media & Storage	Azure Blob Storage (File storage) and Azure CDN (Optimized video delivery).
Payment Processing	Stripe API (Secure checkout integration).
Document Generation	iTextSharp (Dynamic PDF Certificate generation).
Frontend UI/UX	Bootstrap 5, JavaScript, Video.js (Custom player), Progress.js.
Performance Optimization	Implementation of robust caching strategies, database indexing, and automated testing.

5. System Architecture & Design Patterns

The platform is built on a highly scalable, decoupled architecture to ensure maintainability, testability, and future growth.

5.1 N-Tier Architecture

Layer	Responsibility
EduHub.Web (Presentation Layer)	Handles user interface, routing, and HTTP interactions (Controllers & Views).
EduHub.Services (Business Logic Layer)	Encapsulates business rules for course management, enrollments, and progress tracking.
EduHub.Data (Data Access Layer)	Manages database connections, migrations, and data persistence via Repositories.

5.2 Design Patterns

Pattern	Usage in Lumora
Repository Pattern	Abstracts data access. Implemented interfaces include: ICourseRepository, ILessonRepository, IEnrollmentRepository, IProgressRepository, and IReviewRepository.
Unit of Work Pattern	Ensures data integrity by managing multiple transactions as a single atomic unit.
Strategy Pattern	Utilized for calculating varying pricing models and applying dynamic discount strategies.
Observer Pattern	Implemented to track student progress continuously and trigger automated notifications upon lesson/course completion.
Dependency Injection (DI)	Injected throughout the application layers to promote loose coupling, enhanced testability, and modular maintainability.